

#29 · 向量記憶

[未來] 技術代號 D-P1-2 / A-P1-2 · 約 3-4 週

一句話：從查字典升級成聽懂意思

為什麼要修

精靈的記憶是關鍵字標籤搜尋。

現在怎樣

shared/orb-memory.ts 11 種類型 + embeddingVector

欄位（已備未激活）。orbLongTermMemory.search 是 LIKE 文本，TODO 註記需語義搜尋。

問題在哪

問「跟森林有關的」找得到、問「自然系」就找不到。記憶利用率低。

要改什麼

用 OpenAI text-embedding-3-small 把記憶轉成向量、存 Pinecone（最小擾動）、search() 改向量檢索 + 純文本 fallback。

涉及檔案

server/services/memory/embeddingClient.ts（新增）

server/services/orbLongTermMemory.ts（search 改向量檢索）

shared/orb-memory.ts（既有，不改）

drizzle/schema.ts（embeddingVector 已備）

Claude Code 提示詞 (複製貼上)

把下面這段話完整複製，貼到 Claude Code (或 /claude 對話)。Claude Code 會依此完成此項修復。完成後跑下方驗證步驟確認。

請依照計畫附錄 C.1 實作 D-P1-2 / A-P1-2 向量記憶：

季度級工程，3-4 週。

【第 1 部分：embedding client】

1. 開 Pinecone 帳號 (serverless tier 即可)、建一個 index：

- name: healing-studio-memory
- dimensions: 1536 (對應 OpenAI text-embedding-3-small)
- metric: cosine
- cloud: aws / region us-east-1 (或就近)

2. 新增 server/services/memory/embeddingClient.ts：

```
```typescript
export interface EmbeddingClient {
 embed(text: string): Promise<number[]>;
 search(vector: number[], topK: number, filter?: Record<string, unknown>): Promise<MemoryHit[]>;
 upsert(id: string, vector: number[], metadata: Record<string, unknown>): Promise<void>;
}

export class OpenAIPineconeClient implements EmbeddingClient {
 // 用 openai.embeddings.create({ model: "text-embedding-3-small", input: text })
 // 用 pinecone.Index("healing-studio-memory").query({ vector, topK, filter })
}
```
```

3. 從 env var 讀 OPENAI_API_KEY 與 PINECONE_API_KEY。

【第 2 部分：寫入時機】

4. 修改 server/services/orbLongTermMemory.ts:create()，在返回前 fire-and-forget 呼叫 embeddingClient.embed + upsert：

```
```typescript
queueMicrotask(async () => {
 try {
 if (containsSensitiveText(memory.content)) {
 logger.warn("memory_embed_skipped_pii", { id: memory.id });
 }
 return;
 }
}
```

```

}
const vector = await embeddingClient.embed(memory.content);
await embeddingClient.upsert(memory.id, vector, { userId: memory.userId, type: memory.type,
createdAt: memory.createdAt });
await db.update(orbLongTermMemories).set({ embeddingVector: vector
}).where(eq(orbLongTermMemories.id, memory.id));
} catch (err) { logger.warn("memory_embed_failed", { err }); }
});
...

```

### 【第 3 部分：檢索】

- 修改 orbLongTermMemory.search()：先呼叫 embeddingClient.embed(query) 取向量、search top-K（預設 5）；
  - 若 embedding 失敗（API down）降級為純 LIKE 文本搜尋
  - 若無結果，並用文本搜尋作補充

### 【第 4 部分：注入 planner】

- 在 agentPlanner 系統 prompt 組裝處，加入 top-K 相似記憶：

```

```typescript
const memories = await orbLongTermMemory.search(userMessage, 5, { userId });
const memoryContext = memories.map(m => `[過往: ${m.summary}]`).join("\n");
systemPrompt += `\n\n# 相關記憶\n${memoryContext}`;
...

```

【第 5 部分：成本控制】

- 設 monthly cap：在 modelPricing.ts 加 embedding 計費，每月 cap（建議起始 \$50 / 月）。
- 不對 < 50 字的記憶做 embed（沒意義）。

不要做的事：不要遷移到 PostgreSQL pgvector（屬大型 migration）。不要自訓 embedding model。不要把所有對話訊息都 embed（只 long-term memory）。

如何驗證做完了

建一條記憶「使用者偏好森林氛圍 + 暖色調」→ 下次使用者問「我以前喜歡什麼風」→ planner 系統 prompt 看到該條記憶。同樣問「自然系」也能召回（語義近）。Pinecone dashboard 看到 vectors > 0。embedding API 故障時降級到 LIKE 不會 throw。

預估耗時

3-4 週